Computational Economics (2026) 68:3323–3345
https://doi.org/10.1007/s10614-025-11172-z

Examining Challenges in Implied Volatility Forecasting: A Critical Review of Data Leakage and Feature Engineering combined with High-Complexity Models

Argimiro Arratia[1] · **Mahmoud El Daou**[1] · **Jan Kagerhuber**[2] · **Yulia Smolyarova**[2]

Received: 22 December 2024 / Accepted: 28 October 2025 / Published online: 11 November 2025
© The Author(s) 2025

Abstract
We show that a seemingly powerful implied volatility forecasting model, based on neural networks formerly proposed by others, suffers from a data leakage problem due to training with randomized splits of the time-series information causing the model to overfit. Furthermore, we show that the usage of popular performance measures, such as mean square error and gain, can lead to misinterpretations depending on the nature of the target and feature variables. These unintentional errors in data preprocessing, model training and model evaluation have been observed in various published works on forecasting time-dependent variables using machine learning models. The purpose of this work is to show the origin of these errors and how to avoid them, by replicating and critically reviewing one of these machine learning frameworks for modeling implied volatility movements, and along the way propose some solutions. In this way, we hope to contribute to the advancement of reliable and effective volatility forecasting methodologies while fostering a deeper understanding of the associated challenges and opportunities.

Keywords Volatility forecasting · Data leakage · Feature engineering · Implied volatility · Neural networks

1 Introduction

Options are financial instruments that provide the holder with the right, but not the obligation, to buy or sell an underlying asset at a predetermined price (known as the strike price) within a specified period. These instruments are widely used in financial markets for various purposes, including hedging risk and speculation. An option

Mahmoud El Daou, Jan Kagerhuber and Yulia Smolyarova contributed equally to this work.

Extended author information available on the last page of the article

contract typically includes several key parameters: the strike price, expiration date, option type (call or put), and the underlying asset.

Option chains, which represent the collective set of options available for a particular underlying asset, consist of multiple options sharing identical expiration dates. These datasets include critical information such as option Greeks (Delta, Gamma, Theta, Vega), implied volatility (IV) across different days, bid-ask spreads, and other pricing metrics.

Implied volatility (IV) plays a decisive role in options pricing as it represents the market's expectation of future volatility for the underlying asset. Unlike historical volatility, which reflects past price fluctuations, IV captures the collective view of market participants about future price movements. It is a vital input for pricing models like Black-Scholes and serves as an essential indicator for assessing the risk and potential profitability of option positions. Higher implied volatility typically leads to higher option premiums, reflecting the increased uncertainty or potential for significant price swings in the underlying asset. Conversely, lower implied volatility corresponds to lower premiums, indicating a market expectation of relative price stability.

Given its direct impact on option prices, accurately predicting implied volatility is crucial for traders and investors. A reliable IV forecast can enable market participants to anticipate option price movements, allowing them to adjust their positions to exploit market opportunities or manage risk more effectively. Several factors influence implied volatility, with strike price and expiration date being among the most significant. Traders often assess the *implied volatility surface*, a visual representation of IV across various strike prices and expiration dates, to assess market expectations and pricing dynamics. Lower strike prices generally correspond to higher IV, reflecting the market's perception of increased price movement potential. Likewise, options with longer expiration dates tend to have higher IV levels, accounting for the greater uncertainty over extended time horizons.

Various econometric and machine learning methods have been developed to forecast IV dynamics, including vector autoregression (VAR) and time-series analysis (Gonzalves & Guidolin, 2006), decision trees (Vrontos et al., 2021), and neural networks (NN) (Cao et al., 2020; Balaneji & Maringer, 2022). Each approach provides unique insights into the factors influencing IV and equips traders with tools to anticipate market volatility.

This study builds on the work of Jay Cao, Jacky Chen, and John Hull in Cao et al. (2020), to which we will refer to as the CCH model. Our primary objective is to replicate and critically examine the methodology used in the aforementioned paper, with a particular focus on identifying its potential pitfalls such as data leakage during data preprocessing. Furthermore, we propose refinements to the CCH model to improve the accuracy and reliability of IV forecasts. The models we evaluate include: the Hull and White formula (as a baseline model); Neural Networks; and XGBoost, a popular gradient boosting algorithm.

In the context of this study, data leakage –when information from the test set unintentionally influences the model during training– presents a significant risk. We carefully scrutinize the preprocessing steps in the framework proposed in CCH to ensure that such leakage is avoided. By refining data handling procedures, we aim to produce more reliable predictions and better assess model performance.

More specifically, our study extends the CCH model in its 3-feature and 4-feature models for IV forecasting. The 3-feature model uses the daily return of the index, the option's moneyness, and its time to maturity as inputs. We expand this model by adding the VIX index as a fourth feature. The VIX, often referred to as the "fear gauge", reflects market expectations of future volatility derived from S&P 500 index options. Its inclusion enhances the model's ability to capture broader market sentiment and shifts in volatility regimes. However, since the VIX is constant for options on a given day, it requires careful treatment to avoid introducing bias.

To further improve prediction accuracy, we also transformed the regression-based forecasting problem into a classification task. Instead of predicting continuous IV values, we classified IV changes as binary outcomes –either increasing or decreasing. This approach significantly simplified the model's objective and led to better results, as it allowed the model to focus on directionality rather than precise numerical predictions.

The next section provides with a literature review, selecting papers that discuss models similar to those we employ, including the Hull-White model, neural networks, and XGBoost. We then outline our data preprocessing methods, detailing our approach to data cleaning, feature engineering, and target engineering. After replicating the results in Cao et al. (2020) (the CCH model), we critique their methodology, particularly their data handling practices, and discuss potential areas for improvement. Finally, we present our extended models and the results of our experiments. By replicating and extending previous research, we aim to contribute to the development of more effective and trustworthy volatility forecasting models.

2 Literature Review

This section reviews some relevant literature on predicting implied volatility, categorizing key studies into three main perspectives: econometric methods, machine learning approaches, and validation methodologies. This review is far from being exhaustive, as our intention is to highlight both traditional statistical models and modern computational techniques while addressing their potential limitations and the pitfalls researchers may incur into by application of commonly used validation schemes.

Regarding econometric methods for implied volatility forecasting, it is worth mentioning (Gonzalves & Guidolin, 2006), who emphasize the importance of the implied volatility surface (IVS) for pricing, hedging, and trading options. They explore various IVS modeling approaches, distinguishing between cross-sectional and time-series models, with the latter demonstrating greater reliability. Their findings indicate that IVS is statistically predictable, yet real-world frictions such as transaction costs, liquidity constraints, and information asymmetries limit its practical utility. The study suggests that advanced econometric models, including Vector Autoregression (VAR) and Generalized Autoregressive Conditional Heteroskedasticity (GARCH), improve IVS prediction but still face challenges due to market inefficiencies.

A notable econometric contribution is provided by Hull and White (2017), who introduce the Minimum Variance Delta model, which accounts for the correlation

between asset price movements and volatility to optimize hedging performance. Their model outperforms both the Black-Scholes Delta and stochastic volatility models in minimizing variance for S&P 500 options. However, its efficacy diminishes when applied to individual stocks due to increased idiosyncratic noise. This Minimum Variance Delta model will serve as a benchmark in our study.

Wen et al. (2024) present empirical evidence that implied volatility forecasting is primarily driven by historical patterns, with simple linear models such as Ridge regression outperforming more complex nonlinear methods, including deep neural networks and XGBoost. Their findings suggest that traditional econometric approaches remain highly competitive in forecasting implied volatility.

On the side of machine learning approaches to implied volatility forecasting, we have revised the following works. Liu et al. (2019) compare traditional option pricing models with artificial neural networks (ANNs), demonstrating that ANNs can significantly accelerate option pricing and implied volatility computation without sacrificing accuracy. Their findings highlight the efficiency gains machine learning can offer in computational finance.

Zhang and Kline (2007) analyze neural networks (NNs) for quarterly time series forecasting using data from the M3 forecasting competition. They find that data preprocessing, such as detrending and deseasonalization, significantly enhances NN performance. Furthermore, they note that simpler models with fewer variables often generalize better, particularly for macroeconomic datasets.

Vrontos et al. (2021) apply machine learning techniques, including Elastic Net, Gradient Boosting, and Random Forest, to forecast the directional movement of the VIX index. Their results indicate that machine learning models outperform traditional econometric approaches in both statistical accuracy and economic significance.

Balaneji and Maringer (2022) extend machine learning applications to implied volatility forecasting by incorporating sentiment analysis, topic modeling, and XGBoost. Their findings reveal that sentiment-driven features enhance forecasting accuracy, suggesting that textual and alternative data sources can complement financial metrics in volatility prediction.

Finally, Gunnarsson et al. (2024) provide a comprehensive review of machine learning applications in implied volatility forecasting. They conclude that despite the promise of AI/ML techniques, these approaches often fail to significantly surpass traditional econometric models due to methodological limitations and evaluation biases.

This sample of the vast literature highlights the need for caution when adopting the latest AI models for forecasting implied volatility and stochastic time series. Many failures and limitations stem from the misapplication of these models or a lack of understanding of their inner workings, leading to incorrect parameterization and calibration. These issues are particularly problematic when dealing with time-dependent data, often resulting in misleading results.

This concern is particularly acute when such findings are disseminated through peer-reviewed publications, potentially misguiding subsequent research and practical applications. A pervasive challenge across the literature on applying machine learning to time-dependent data stems from the potential for methodological oversights, particularly concerning validation strategies. As discussed in foundational works such as Bergmeir and Benítez (2012), Schnaubelt (2019) and (Lopez de Prado, 2018,

Ch. 7) among others, standard cross-validation techniques, which involve random shuffling of data, are fundamentally ill-suited for time series due to the inherent temporal dependencies and potential for data leakage. Numerous studies implicitly fall into this trap by either one (or all) of the following reasons: employing default cross-validation schemes in popular libraries without adjusting for temporal order; reporting exceptionally high performance metrics (R^2 close to 1, very low MAE/RMSE) on training or validation sets that strongly suggest overfitting, especially in domains like financial forecasting where such accuracy is highly improbable in true out-of-sample scenarios; failing to explicitly detail their train-test split methodology, leaving room for assumptions that often lead to non-temporal data partitioning.

Taken together, these facts emphasize the importance of robust evaluation methodologies in volatility forecasting research. Although machine learning models offer potential advantages, their predictive performance is highly dependent on appropriate validation frameworks. One of the purposes of this paper is to call attention to these fine but important details in the application of ML models, as well as the need to employ robust validation strategies to ensure reliable and interpretable forecasting results.

3 Models

In this section, we explain the three models we trained to produce predictions: Hull and White model as a benchmark, neural networks, and XGBoost.

3.1 Black and Scholes

The Black-Scholes model is a widely used options pricing model that provides a theoretical framework for valuing European-style options. In our study, we employ the practitioner Black-Scholes model, where the volatility parameter in the Black-Scholes formula is replaced by the implied volatility. One of the key metrics derived from this model is the option's Delta (Δ), which measures the sensitivity of the option price to changes in the price of the underlying asset. Delta is calculated as the partial derivative of the option price with respect to the underlying asset price. The formula for Delta in the practitioner Black-Scholes model remains consistent with the traditional Black-Scholes formula:

$$\Delta = e^{-qT}\Phi\left(\frac{\ln(S/K) + (r - q + \frac{\sigma^2}{2})T}{\sigma\sqrt{T}}\right),$$

where Φ is the cumulative standard normal distribution function, S represents the current price of the underlying asset, K is the strike price of the option, r is the risk-free interest rate, σ is the implied volatility of the underlying asset, T is the time to expiration of the option, and q is the dividend yield of the underlying asset.

Delta values measure the sensitivity of option prices to changes in the price of the underlying asset and serve as a proxy for the moneyness of the option. For call options, Delta values range between 0 and 1, while for put options, they range

between -1 and 0. A Delta close to 1 for call options (or close to -1 for put options) suggests that the option is deep in-the-money, meaning that it closely tracks changes in the underlying asset's price. Conversely, a Delta close to 0 indicates that the option is far out-of-the-money, implying lower sensitivity to changes in the underlying asset price. Understanding Delta values provides insights into the relative moneyness of options and aids traders in assessing the risk-return profile of their option positions.

3.2 Hull and White

We use the Hull and White formula as a benchmark in our study, similar to the approach taken in the CCH model. Proposed by Hull and White (2017), this formula considers minimum variance Delta estimates and offers an analytical model for estimating implied volatility changes ($y_{t,o}$) based on the changes of return of the index ($\mathrm{SPY_ret}_{t-1}$), time to maturity ($\mathrm{DTE}_{t-1,o}$), and practitioner Black-Scholes Delta ($\Delta_{t-1,o}$). The formula is as follows:

$$\mathbb{E}(y_{t,o}) = \mathrm{SPY_ret}_{t-1} \cdot \frac{a + b \cdot \Delta_{t-1,o} + c \cdot \Delta_{t-1,o}^2}{\sqrt{\mathrm{DTE}_{t-1,o}}}.$$

This model captures the relationship between the index return, option Delta, and days to maturity in predicting the change in implied volatility for each option. The parameters a, b, and c are estimated from the data.

This model was then used to evaluate the performance of the main models, two neural networks: the first utilizing only three features ($\mathrm{SPY_ret}_{t-1}$, $\Delta_{t-1,o}$, $\mathrm{DTE}_{t-1,o}$), and the second including $\mathrm{VIX}_{t-1,o}$ as a fourth input parameter.

3.3 Neural Network

Neural networks have emerged as powerful tools in financial modeling due to their ability to capture complex, non-linear relationships within data. At their core, neural networks consist of layers of interconnected nodes, or neurons, each of which processes input data through a series of weights and activation functions to produce an output.

A neural network typically includes an input layer, one or more hidden layers, and an output layer. Each neuron in a layer receives inputs from the neurons in the previous layer, processes these inputs using a weighted sum and an activation function, and passes the result to the neurons in the subsequent layer. Common activation functions include the sigmoid, tanh (hyperbolic tangent function), and ReLU (Rectified Linear Unit), each contributing differently to the learning dynamics of the network. The network learns by adjusting the weights through a process called backpropagation, which minimizes the error between the predicted and actual outputs using gradient descent.

In our study, we implemented a neural network model to forecast changes in implied volatility with a similar parameterization as used in the CCH model to exactly reproduce their results. We adopted a standard feed-forward neural network

architecture with three layers, each comprising 80 neurons, and utilizing a sigmoid activation function.

Training the neural network involved iterating over the dataset for 40 epochs. An epoch represents one complete pass through the entire training dataset, allowing the network to learn and adjust its weights progressively. We used a supervised learning approach where the network learns to map input features, such as Delta, days to expiration, S&P 500 index return, and VIX value, to the target variable, which is the change in implied volatility.

The authors of the CCH model reasoned that the chosen hyperparameters of three layers with 80 neurons each, sigmoid activation, and 40 training epochs yielded the best predictions. This assessment was subjectively evaluated based on the smoothness of the predicted volatility surface.

3.4 XGBoost

XGBoost is an efficient and scalable implementation of gradient boosting framework. In this study, we utilized XGBoost to train gradient boosting models for volatility forecasting. XGBoost operates by sequentially building an ensemble of weak learners, typically decision trees, where each successive tree attempts to correct the errors of its predecessors. This method is particularly powerful for structured data and can handle both regression and classification tasks effectively.

We trained XGBoost models with the same feature sets as the neural networks: two-feature (SPY_ret, Delta), three-feature (SPY_ret, Delta, DTE), and four-feature (SPY_ret, Delta, DTE, VIX). The models were evaluated using the same performance metrics to ensure a consistent comparison across different modeling approaches. The hyperparameters for XGBoost were optimized to balance model complexity and performance.

4 Methodology

Our methodology involved several key steps to investigate volatility forecasting challenges, focusing on data collection, preprocessing, model training, and performance evaluation. We constructed a dataset using option data from OptionsDX[1] (2010-2018) as this is the data used for the CCH model. The dataset included implied volatility (IV), Delta, days to expiration (DTE), S&P 500 index return (SPY_ret), and VIX values. The target variable was the change in IV, with additional exploration of log returns and binary classification of IV movements. Data cleaning filtered for options with over 14 days to expiration, Delta between 0.05 and 0.95, and call options only. For model training, we used the Hull and White formula as a baseline, alongside a feedforward neural network (three layers, 80 neurons each, trained for 40 epochs) and the XGBoost algorithm. We evaluated models with different feature sets (2-feature: SPY_ret, Delta; 3-feature: SPY_ret, Delta, DTE; 4-feature: SPY_ret, Delta, DTE, VIX). To avoid data leakage, we implemented a chronological train-val-

[1] https://www.optionsdx.com/

idation-test split, ensuring options from the same day were grouped together. Performance evaluation used Mean Squared Error (MSE) and weighted accuracy (based on IV log differences). We compared neural networks and XGBoost models against the baseline, also transforming the regression problem into a classification problem for IV movement direction. Identified potential pitfalls included data leakage, addressed by chronological splitting, and the importance of target variable choice, with log returns and binary classification providing better results. We performed robustness checks by applying models to an additional test set (post-2018) to ensure generalization to unseen data. This structured approach aimed to enhance model reliability and performance in volatility forecasting.

5 Data Preparation

The initial objective of our study was to validate and reproduce the findings reported for the CCH model. However, we encountered significant challenges due to the unavailability of the specific option data utilized by the original researchers. To address this, we employed alternative methods to construct a comparable dataset. This involved extensive data scraping and formatting procedures to mimic the structure and characteristics of the original dataset as closely as possible. Specifically, we utilized option chain data from OptionsDX, covering the period from 2010 to 2018. It is worth noting that while the CCH model used data from 2010 to 2017, we extended our dataset by an additional year. This extra year was later utilized as an unseen test set to evaluate the generalization capability of our models.

From the raw option chain data there were several additional steps we took to recreate CCH dataset as closely as possible: First, the option chain data was grouped to form separate time series containing Delta (Δ), Days to Expiry (DTE) and Implied Volatility for every unique option. Furthermore, for each day $t_1, t_2, t_3, \ldots$, we also observe the S&P 500 index return and VIX value, which are recorded in the last row of the table (Table 1).

Each option ($o_1, o_2, \ldots, o_N \in O$) is uniquely defined by its expiration date and strike price and forms a time series of observations at days $t_1, t_2, t_3, \ldots$. This definition is critical as it ensures that each option is distinctly identified for accurate tracking and analysis over time. Each observation includes Delta, time to maturity, and IV

Table 1 Raw Data structure

	t_1	t_2	t_3
o_1	$\Delta_{t_1,o_1}; \mathrm{DTE}_{t_1,o_1}; \mathrm{IV}_{t_1,o_1}$	$\Delta_{t_2,o_1}; \mathrm{DTE}_{t_2,o_1}; \mathrm{IV}_{t_2,o_1}$	
o_2	$\Delta_{t_1,o_2}; \mathrm{DTE}_{t_1,o_2}; \mathrm{IV}_{t_1,o_2}$	$\Delta_{t_2,o_2}; \mathrm{DTE}_{t_2,o_2}; \mathrm{IV}_{t_2,o_2}$	
o_N	$\Delta_{t_1,o_N}; \mathrm{DTE}_{t_1,o_N}; \mathrm{IV}_{t_1,o_N}$	$\Delta_{t_2,o_N}; \mathrm{DTE}_{t_2,o_N}; \mathrm{IV}_{t_2,o_N}$	
S&P	$\mathrm{SPY_ret}_{t_1}$	$\mathrm{SPY_ret}_{t_2}$	
VIX	VIX_{t_1}	VIX_{t_2}	

(which is later transformed into the target variable by taking first differences). Note that the strike price and expiration date are not included as input variables.

After restructuring the data, we proceeded to create the same features and targets as for the CCH model, ensuring consistency and comparability in our results. The detailed steps involved in the preprocessing phase are outlined below.

5.1 Target Engineering

The target variable, $y_{t,o}$ represents the change in implied volatility (IV) for each specific option $o \in O$ on each day t, calculated as $y_{t,o} = \mathrm{IV}_{t,o} - \mathrm{IV}_{t-1,o}$. Here, $\mathrm{IV}_{t,o}$ denotes the implied volatility of option o on day t and $\mathrm{IV}_{t-1,o}$ represents the implied volatility of the same option on the previous day $t-1$. In particular, not all options have a corresponding value y for each day t, and similarly not every day t has values y for all options o. This is mainly due to the fact that our dataset spans a large time frame, and once an option expires, there are no further data points for that option.

5.2 Feature Engineering

The input features $X_{t,o}$ to predict $y_{t,o}$ include several key financial metrics:

- Delta ($\Delta_{t-1,o}$) of option $o \in O$: the measure of moneyness of the option
- Days to expiration ($\mathrm{DTE}_{t-1,o}$): the remaining time until the option's expiration
- S&P 500 index return ($\mathrm{SPY_ret}_{t-1}$): the return of the index from the previous day
- VIX value (VIX_{t-1}): for the four-feature model, the VIX value from the previous day is also incorporated, providing an additional measure of market volatility

The distributions that these features form can be seen in Fig. 1.

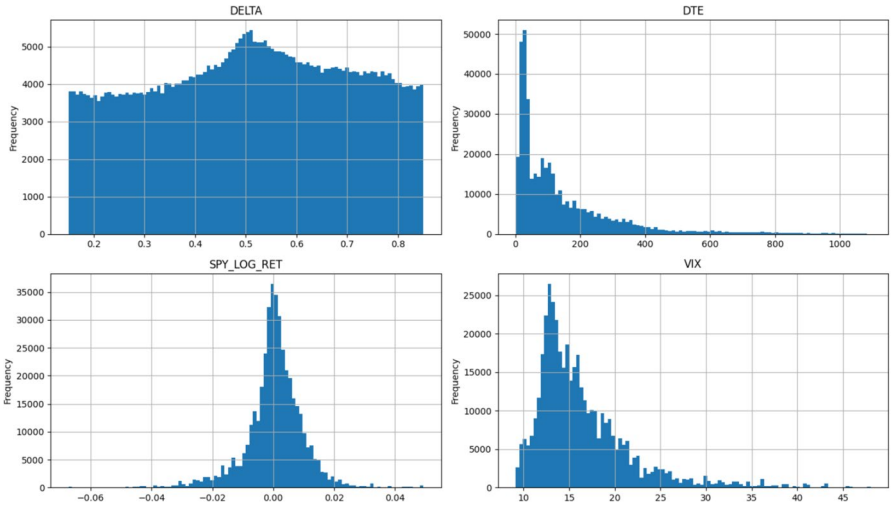


Fig. 1 Feature Distribution

These distributions indicate that the scale of the features vary widely, suggesting the necessity for standard scaling before training our neural networks. It is also notable that for all options observed on the same day, the index return and VIX value are identical, as these features depend on general market conditions rather than specific options.

5.3 Data Cleaning

Following the calculation of dependent and independent variables, we applied a data cleaning process mirroring that detailed in the CCH model. Given the inherent noise in IV calculations, particularly for far out-of-the-money options or those nearing expiration, the following filtering criteria were applied:

- Days to Expiration: Only options with more than 14 days until expiration were included to reduce volatility spikes near maturity. As expiration approaches, Vega decreases—reducing the option's sensitivity to IV changes—while Gamma increases, causing rapid shifts in Delta. With the option's price dominated by intrinsic value and minimal time value, even small price changes can lead to disproportionately large swings in implied volatility.
- Option Delta: Options with a Delta between 0.95 and 0.05 were selected. This range effectively filters out deep in-the-money and far out-of-the-money options, focusing on those with more stable pricing characteristics.
- Call Options: Data was restricted to call options (options with positive Delta), ensuring consistency in the type of contracts analyzed.

By adhering to these preprocessing steps, we aimed to faithfully reproduce the conditions and assumptions of the original study, allowing for a robust comparison of our findings with those reported by the CCH model.

Our final constructed dataset comprises of 427,000 observations. Despite our meticulous efforts in data collection and preprocessing, the resulting dataset has a slightly reduced sample size compared to the original. Nonetheless, given the constraints faced, this outcome represents the best approximation achievable within the scope of our study.

6 Replication

The replication of the original CCH model involved several critical steps to ensure the fidelity and accuracy of our results. This section outlines our approach, from the data partitioning strategy to the evaluation of model performance, and presents a comparative analysis of our findings against those reported in the original study.

The authors opted for a train-validation-test split of 70/20/10, a widely adopted strategy in machine learning, particularly suited to the dataset's size. Additionally, they followed a common practice of shuffling the data before partitioning. This as we shall demonstrate is a cause for data leakage and consequently model overfitting.

Table 2 Original Study Results

Model	MSE	Gain
Benchmark	0.0001102	-
NN_3	0.0000984	0.1072
NN_4	0.0000372	0.6212

In evaluating model performance, the authors focused on the metric of gain, which provides a comparative measure of improvement over a baseline (Hull and White's formula). Gain is computed based on sum of square errors as:

$$\text{Gain} = 1 - \frac{\text{SSE(Model A)}}{\text{SSE(Model B)}}.$$

The CCH modeling results are reproduced in Table 2 for the sake of comparison. Through their analysis, the authors observed a significant gain over the baseline, with neural networks utilizing three features exhibiting a gain of 10.72% with a mean squared error of 0.0000984, while those employing four features demonstrated a gain of 62.12% with a mean squared error of 0.0000372. From those numbers we were also able to calculate the mean squared error of their benchmark model which equates to about 0.0001102.

Our subsequent steps involved replicating the target engineering, feature engineering, and data cleaning processes as described earlier. Then, we constructed the benchmark model and implemented the two neural networks with varying feature sets, comprising 3-feature and 4-feature configurations. Furthermore, to broaden the scope of our comparison, we trained a gradient boosting model using the XGBoost algorithm on the same features. Additionally, we included simpler models containing only two features: S&P 500 return and Delta. Upon completion of these analyses, the obtained results closely mirrored those reported in CCH. The Mean Squared Error (MSE) shown in Table 3, encapsulates the quantitative outcomes of our investigations.

Upon comparative analysis, our benchmark model demonstrated a marginal performance edge over the authors', hinting at potential differences in our datasets. However, the overall results remain closely aligned. Notably, our three-feature neural network exhibited a slightly higher gain than theirs, achieving a gain of 12.67%, while our four-feature neural network fell short in achieving similar gains, having a gain of 45.75%. This discrepancy is likely influenced by dataset disparities and the superior performance of our benchmark model. Furthermore, it is noteworthy that our

Table 3 Model Results on Test Set

Model	MSE	Gain over Benchmark	R-Squared
Benchmark	0.00001045	-	0.0028
NN_2	0.00000927	0.0901	0.0926
NN_3	0.00000890	0.1267	0.1292
NN_4	0.00000551	0.4575	0.4609
XGB_2	0.00000931	0.0862	0.0888
XGB_3	0.00000668	0.3439	0.3466
XGB_4	0.00000342	**0.6638**	0.6648

XGBoost models demonstrated gains of 34.39% and 66.38%, significantly outperforming their neural network counterparts, with no hyperparameter tuning required.

The findings from both their study and our replication indicate the potential for substantial predictive gains through the utilization of higher-complexity models, such as neural networks, in forecasting changes in implied volatility. Particularly impressive is the R-squared performance of the high-complexity models, which stands in stark contrast to the benchmark model's R-squared of 0.0028, suggesting minimal improvement over a simplistic mean-based approach. Achieving an R-squared of 0.4609 with only a four-feature model appears almost too good to be true. Indeed, there are notable concerns and limitations within this study that warrant further exploration.

7 Critique

Despite successfully replicating the results in Cao et al. (2020), we believe there are significant omissions and issues that need to be addressed. In this section, we will discuss these concerns in detail. The main problem we identified is the potential for data leakage due to random data shuffling, which can lead to an overestimation of the model's predictive power. We will analyze this issue in the next subsection.

One notable issue is that the gain of a model is intricately tied to the performance of the baseline. In our scenario, the baseline model achieves an R-squared of 0.003 (0.3% explained variance), a figure that warrants attention. In our assessment, it is imperative to consider this factor when discussing the concept of gain.

7.1 Data Segmentation

In predictive modeling, dividing the dataset into distinct subsets serves several crucial purposes, including model training, hyperparameter tuning, and performance evaluation. The primary subsets commonly employed are the training set, the validation set, and the test set, each serving a specific role in the model development process.

The training set forms the foundation of model development, used to train the model on patterns and relationships present in the data. The validation set is essential for model selection and hyperparameter tuning, evaluating the model's performance on unseen data, and enabling iterative adjustments for optimal performance. The test set serves as the final benchmark, providing an unbiased estimate of the model's ability to generalize to new data, ensuring reliable real-world performance metrics.

Randomly dividing the dataset into these subsets is common practice in machine learning due to several advantages. These include representativeness, ensuring each subset contains a representative sample of the overall dataset; generalization, allowing the model to generalize to unseen data; unbiased evaluation, providing objective metrics of model performance; and reproducibility, ensuring results are consistent across different runs.

In our study, we encountered a unique challenge related to data leakage due to the nature of our dataset and the task of volatility forecasting when applying random

splitting. There are two main properties of our data that contribute to potential data leakage.

First, options with similar expiration dates and strike prices are likely to be highly correlated. For example, assume options $o, o' \in O$ characterized by the same expiration date and similar strike prices. Then time-series of IV changes $y_{t_1,o}, y_{t_2,o}, y_{t_3,o}, \ldots$ and $y_{t_1,o'}, y_{t_2,o'}, y_{t_3,o'}, \ldots$ might be similar. This is particularly true if the expiration is far into the future.

Second, our dataset has the property that features like the SPY index return and the VIX index remain constant for all options from the same day. This means the model can inadvertently learn date information, allowing it to exploit patterns specific to dates rather than the underlying relationships we aim to capture.

The issue arises from the potential for unintentional information leakage during the data splitting process. Specifically, if we were to employ a random train-test split approach, there's a risk that options from the same day, characterized by the same VIX index, the same return on the index, and similar changes in implied volatility, may end up in both the training and test sets. This scenario can lead to biased model performance evaluation and misleading results.

Given the time-series structure of our data, it was imperative to devise a proper data splitting strategy that preserves temporal dependencies. Specifically, we ensured that all options from one day were grouped together in one set (either train, validation, or test). This approach prevents the model from learning spurious correlations based on the date, thereby mitigating the risk of data leakage. Furthermore, we adhered to a chronological split, where earlier data was used for training and later data for testing. This ensured that the model's performance was evaluated on unseen future data, providing a more realistic assessment of its predictive capabilities.

Having described the potential issue of data leakage due to the temporal aspects of our features, we now aim to investigate whether this problem is indeed affecting our model. Specifically, we will analyze if options from the same day have similar target values and exhibit high correlations with each other. By doing so, we can determine the extent to which the date information might be influencing our model's performance.

The observations in Table 4 underscore a critical aspect of the dataset: the implied volatility (IV) change tends to be quite similar across most samples. Additionally,

Table 4 10 Dataset Samples for day February 5th 2018

EXPIRY	STRIKE	IV	DELTA	DTE	SPY_RET	VIX	$Y_{o,t}$
2018-02-16	249.00	0.618	0.708	11	-0.0427	37.32	-0.1784
2018-02-16	257.00	0.527	0.612	11	-0.0427	37.32	-0.1601
2018-02-16	257.50	0.520	0.604	11	-0.0427	37.32	-0.1704
2018-02-16	258.00	0.510	0.598	11	-0.0427	37.32	-0.1615
2018-02-16	259.00	0.480	0.585	11	-0.0427	37.32	-0.1386
2018-02-16	261.00	0.468	0.549	11	-0.0427	37.32	-0.1448
2018-02-16	262.50	0.442	0.522	11	-0.0427	37.32	-0.1411
2018-02-16	263.00	0.458	0.513	11	-0.0427	37.32	-0.1462
2018-02-16	264.00	0.425	0.491	11	-0.0427	37.32	-0.1222
2018-02-21	271.00	0.347	0.315	16	-0.0427	37.32	-0.1096

it's noteworthy that the VIX and SPY return remain constant for all t. While this alone might not pose a problem, the issue arises when these different samples are shuffled into the train and test sets. This scenario presents a case of data leakage. Consequently, it becomes relatively straightforward for high-dimensional models like neural networks or decision trees to extrapolate that samples with the same SPY return, VIX, and days to maturity (DTM) will likely have similar targets, as they all occurred on the same day.

This scenario leads to remarkably good 'out of sample' performance. However, this performance wouldn't necessarily translate to real out-of-sample data comprising entirely new days, as the model wouldn't have access to the same-day data at that time. Therefore, utilizing neural networks to predict changes in IV in real-world live settings would likely yield poor results, despite the seemingly impressive out-of-sample performance. This situation exemplifies a classic case of data leakage, precisely what is occurring in this study.

To confirm this hypothesis, we employed an additional test set consisting solely of the most recent trading days post-2018, a time frame the model had not been trained on. Interestingly, in this test set, the constraint baseline model outperformed both the neural networks and Gradient Boosting models by a large margin.

We can see that, when evaluating these models on a new test set composed of entirely new days, the results were markedly poor. As shown in Table 5, all models exhibited extremely poor performance on unseen data, indicating a failure to generalize due to incorrect data shuffling in earlier evaluations. This emphasizes the critical need for proper data handling practices, including robust train-validation-test splitting, to ensure models can effectively generalize and perform reliably in real-world volatility forecasting applications. Importantly, these models demonstrated worse performance than the baseline model on the new test set, further highlighting the detrimental impact of data leakage and the necessity for rigorous model validation procedures.

Interestingly, we can see that the 2-feature models, which exclude the days-to-expiration variable, perform better than the more complex 3- and 4-feature models on the new test set. However, their results still fall short compared to the benchmark model and are significantly worse than the results on the original test set (Table 3). This indicates that removing the days-to-expiration variable does not fully mitigate the data leakage problem, highlighting that the issue extends beyond just this single feature.

Table 5 Model Results on Unseen Test Set

Model	MSE	Gain over Benchmark	R-Squared
Benchmark	0.000080	-	0.0454
NN_2	0.000085	-0.0669	-0.0184
NN_3	0.000093	-0.1643	-0.1115
NN_4	0.000122	-0.5247	-0.4555
XGB_2	0.000093	-0.1637	-0.1109
XGB_3	0.000116	-0.4517	-0.3858
XGB_4	0.000102	-0.2845	-0.2262

Later in this study, we will delve into strategies for mitigating data leakage, as well as techniques for fitting and assessing the performance of more complex models without compromising data integrity.

We want to ensure that for any day t such that there is an option o observed on this day in the training set there is no option o' observed at this day in the validation or testing set:

$$\forall t : \exists o \in O \text{ s.t. } y_{t,o} \in \text{Train} \implies \nexists o' \in O \text{ s.t. } y_{t,o'} \in \text{Test} \cup \text{Valid}.$$

To address the issue of data leakage, we implemented a rigorous train-validation-test splitting strategy. Specifically, all options observed on earlier dates were included in the training set, while options from more recent dates were partitioned into the validation and test sets. This approach ensures that all options from the same day are grouped together and assigned exclusively to either the training, validation, or test set. By doing so, we prevent the model from inadvertently learning from temporal information embedded in features like the VIX index and S&P 500 returns, thereby mitigating the risk of data leakage.

After applying this refined splitting strategy, we re-fit and re-evaluated our models' performance using the same neural networks and XGBoost models previously employed. Table 6 presents the results on the test set under the corrected methodology:

The results from Table 6 reveal that all models trained on the corrected data splitting strategy demonstrate nearly identical mean squared error (MSE) values. Specifically, the MSE differences between these models are minimal, resulting in gains over the benchmark model not exceeding 1.5%. These findings suggest that, with the selected input features, the neural networks and Gradient Boosting Models tested in this study do not substantially outperform the Hull and White formula.

To ensure that we correctly identified the root of the problem, we also included the results of testing models fitted with correct splitting on our additional test set in Table 7. If we have effectively fixed the data leakage problem, we should not see any dramatic deterioration of the results compared to the results on the original test set, as was observed in Table 5 compared to Table 3.

Based on the comparison of model performance results fitted with random and chronological splitting and tested on both the original and additional test sets, we can conclude that the impressive results initially obtained were caused by a data leakage problem. When employing the correct data splitting strategy, these models do not substantially outperform the benchmark model. This highlights the critical

Table 6 Model Results on Test Set with Corrected Split

Model	MSE	Gain over Benchmark	R-Squared
Benchmark	0.000082	-	0.0025
NN_2	0.000082	0.0102	0.0126
NN_3	0.000081	0.0154	0.0178
NN_4	0.000083	-0.0123	-0.0097
XGB_2	0.000082	-0.0000	0.0025
XGB_3	0.000082	0.0001	0.0026
XGB_4	0.000083	-0.0078	-0.0052

Table 7 Model Results on Unseen Test Set with Corrected Split

Model	MSE	Gain over Benchmark	R-Squared
Benchmark	0.0000898	-	0.0218
NN_2	0.0000904	-0.0061	0.0158
NN_3	0.0000935	-0.0376	-0.0135
NN_4	0.0000910	-0.0136	0.0085
XGB_2	0.0000934	-0.0306	-0.0115
XGB_3	0.0000909	-0.0098	-0.0061
XGB_4	0.0000892	0.0066	0.0282

importance of proper data handling to ensure the validity and reliability of predictive models in real-world applications.

An important takeaway from this analysis is the necessity for caution when constructing new datasets for benchmarking complex model architectures. Creating valid datasets from real-world scenarios presents its own set of challenges, and errors in this phase can undermine the credibility of the study's findings. When the primary focus is on refining model architectures and comparing machine learning methodologies, a better approach may involve utilizing pre-existing, validated datasets or resorting to synthetic data generation. These alternatives offer a more robust foundation for experimentation, minimizing the risk of inadvertent biases or data leakage that could skew the results.

7.2 Choice of Target

When developing predictive models for volatility forecasting, the selection of the target variable significantly influences model performance and applicability. In this subsection, we explore the limitations associated with using the change in implied volatility (IV) as a target variable in regression tasks, particularly its interaction with the Mean Squared Error (MSE) loss metric. This exploration prompts consideration of classification tasks as an alternative approach to effectively capture and predict market movements.

Considering the change in implied volatility (IV) as the target, a certain bias is introduced. Samples with higher IV exert a disproportionate influence on the total errors (MSE) compared to changes in options with initially low IV. This introduces a bias in the model's learning process, potentially skewing the predictive outcomes.

For example, consider an option with a very high IV of 1.2, which subsequently decreases to 1.15. The change in IV (the target variable y for this option) would be -0.05. In contrast, if we have an option with a more typical IV of 0.2, and the IV increases to 0.22, the change in IV would be only $+0.02$. However, in percentage terms, the price of the option changes significantly more in the second case than in the first. Consequently, our machine learning algorithms place greater importance on accurately predicting high IV samples, despite the fact that, from a financial perspective, changes in lower IV samples are equally as relevant. After all, it's the percentage change of IV that ultimately impacts the price of our portfolio.

In the subsequent section, we introduce a modification where we consider the difference in the natural logarithm of IV

$$log_ret_{o,t} = \ln(IV_{t,o}) - \ln(IV_{t-1,o}) = \ln\left(\frac{IV_{t,o}}{IV_{t-1,o}}\right),$$

thereby accounting for relative changes rather than absolute changes in IV.

Another noteworthy issue arises from treating the problem solely as a regression task. This approach introduces a phenomenon where predictions in the correct direction, which would potentially result in profit due to the movement of option prices aligning with our predictions, still incur a higher MSE error compared to predictions in the wrong direction but closer to zero. This discrepancy can distort the evaluation of model performance, potentially overlooking the efficacy of predictions that accurately forecast market movements.

For instance, consider two distinct samples: Sample 1 experiences a significant decrease in implied volatility (IV) by $+0.2$, while Sample 2 witnesses a minor increase in IV by -0.02. Now, suppose our model predicts a decrease of -0.02 for both samples.

The squared residuals for each sample would then be calculated as follows:

$$\begin{aligned}
\text{MSE}_1 &= (y_{pred} - y_1)^2 = (-0.2 - (-0.02))^2 = 0.18^2 = 0.0324 \\
\text{MSE}_2 &= (y_{pred} - y_2)^2 = (0.02 - (-0.02))^2 = 0.04^2 = 0.016
\end{aligned}$$

So, in essence, our algorithm perceives Sample 1 as a significantly worse prediction, despite correctly forecasting the direction of change in IV, which would have resulted in profitable trading. Conversely, for Sample 2, the model recommends a direction opposite to what would have been financially beneficial, leading to potential losses. This disparity underscores a fundamental issue with error metrics like MSE and regression analysis in general when applied to this context. Sample 1 represents a superior prediction compared to Sample 2, yet the error metrics fail to accurately reflect this distinction. This prompts a critical examination of whether regression and MSE are indeed suitable approaches for addressing problems of this kind.

In the following section we will explore an alternative approach by considering a classification task instead. This approach would aim to predict the direction of IV change (increase or decrease) rather than the exact magnitude, potentially providing clearer insights into market movements and facilitating more robust trading strategies.

8 Enhanced Methodology

Having addressed the data leakage issue, we now turn our attention to finding a more appropriate metric for evaluating our model's performance. Previously, we discussed why regression and Mean Squared Error (MSE) might not be the best measures of a model's effectiveness in this context. Consequently, we suggested transforming the problem into a classification task. However, this approach comes with its own set of challenges.

Simply converting our regression problem into a classification problem as defined in Eq. 1 and measuring model performance based on classification accuracy could introduce a different bias into our models that needs to be addressed.

$$y_{o,t} = \begin{cases} 1 & \text{if } log_ret_{o,t} > 0 \\ 0 & \text{if } log_ret_{o,t} <= 0, \end{cases} \tag{1}$$

For instance, if we have one sample where IV increased by a large amount (0.2) and another sample where IV only increased by a marginal amount (0.01), converting these samples into classes and using metrics like accuracy or log loss would place equal importance on classifying both samples correctly. However, correctly classifying the first sample is far more critical for our purpose.

To address this issue, we propose to use Weighted Accuracy as our primary metric. Weighted Accuracy incorporates sample weights w along with the True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN) to compute a more balanced measure of accuracy. Specifically, Weighted Accuracy accounts for the proportion of correctly classified samples while considering the importance of each sample:

$$\text{Weighted Accuracy} = \frac{w_p \cdot TP + w_n \cdot TN}{w_p \cdot (TP + FN) + w_n \cdot (TN + FP)}, \tag{2}$$

With w_n and w_p being the total weight of negative and positive predictions respectively. The sample weights are calculated using the absolute value of the implied volatility log difference:

$$weight_{o,t} = |log_ret_{o,t}|\,. \tag{3}$$

By doing so, we ensure that samples with more significant changes in IV are given greater importance in our model evaluation. Note that using logarithm differences allow us to account for relative changes rather than absolute changes in IV. In this context, when using the weighted accuracy to analyze our models, it can be interpreted as the ratio of total correctly classified implied volatility change (log difference). This metric should translate much better to actual portfolio performance when trading options.

With an improved method for comparing model performance in place, we proceeded to build another set of different models for comparison. In addition, we converted the predictions of the previous models scored with MSE to binary classification as follows:

$$class_{o,t} = \begin{cases} 1 & \text{if } regression_pred_{o,t} > 0 \\ 0 & \text{if } regression_pred_{o,t} \leq 0. \end{cases} \tag{4}$$

After that we created a new target $log_ret_{o,t} = \ln(IV_{t,o}) - \ln(IV_{t-1,o})$ and used that in addition to the binary classification target Eq. 1 to train new neural networks and gradient boosted trees on the same feature set as used before. Moreover we intro-

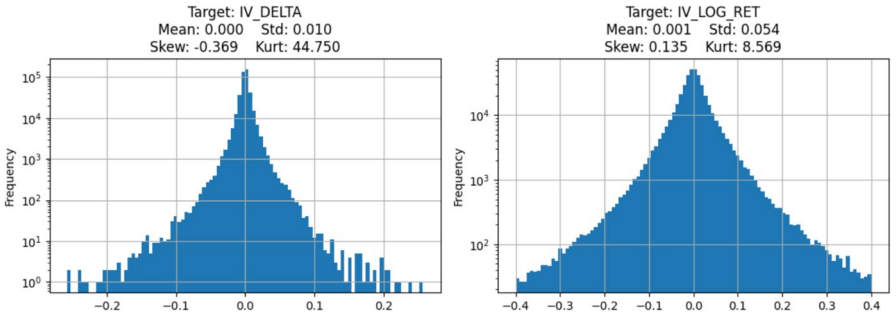


Fig. 2 Target Distribution

duced a 5 Factor Model, that uses $IV_{o,t-1}$ as an additional feature, as we expected it might further improve the model performance.

As illustrated in Fig. 2, the new log_ret target (right) exhibits significantly fewer large outlier samples compared to our previous target (left), as shown by the kurtosis value. We anticipate that this reduction in outliers will lead to a more robust final model when using Mean Squared Error as the loss function.

Table 8 shows the results of all models that where build with no data leakage in training and testing sets.

It's important to note that always predicting a down move in implied volatility (which is the most common label) for all samples (i.e. $\forall x(y_{pred}(x) = 0)$) yields a weighted accuracy of 0.52096. Hull & White's baseline model managed to achieve a better result than that with 0.55942.

With that in mind, it's clear that the 2-Factor Neural Network and the 5-Factor XGBoost model trained on the true difference in implied volatility as a target are the worst-performing models in the analysis.

An interesting observation from these results is that using the same input features but with log difference as the target leads to a decent improvement in weighted accuracy. Since these results are achieved on true out-of-sample data, this reinforces our thesis that log returns of the implied volatility is a better target for our task.

In general, models trained on the binary classification target performed the best, with the XGBoost model trained on four features achieving the highest weighted accuracy score of 0.6191, closely followed by the 5-Factor Neural Network and the 3-Factor XGBoost model.

Table 8 Out of Sample Weighted Accuracy

Target	Model	2 Factors	3 Factors	4 Factors	5 Factors
true diff	NN	0.5378	0.5508	0.5622	0.5590
	XGB	0.5562	0.5470	0.5091	0.4785
log diff	NN	0.5536	0.5469	0.5499	0.5735
	XGB	0.5441	0.5603	0.5726	0.5314
binary	NN	0.5580	0.5562	0.5697	0.5802
	XGB	0.5725	0.5941	**0.6191**	0.5551

Another notable detail in our results is that adding features decreases the performance of the XGBoost models in most cases, whereas it often helps increase the performance of our neural networks.

As a summary, to better evaluate models predicting changes in IV, we transformed the regression problem into a classification task, using a piecewise function to classify log returns of IV as positive or negative. Given the unequal importance of correctly classifying significant changes, we adopted Weighted Accuracy as our primary metric, using sample weights based on the absolute value of log differences.

Compared to the results obtained using MSE as our metric (as shown in Table 3), employing weighted accuracy greatly enhances our ability to interpret our models. It allows us to discern meaningful differences in performance that were previously unclear.

Additionally, classification problems are inherently easier to solve than regression tasks, and that weighted accuracy better aligns with the real problem we are trying to solve, we are gaining a double benefit from using this new metric.

Furthermore, achieving a 61% accuracy with only 4 input features is quite impressive. We were pleasantly surprised by our model's performance compared to the benchmark set by the CCH model. However, it's worth noting that Hull and White's model also demonstrates respectable performance.

9 Limitations

While our study provides valuable insights into the challenges and potential pitfalls of volatility forecasting, it is important to acknowledge its limitations. These limitations highlight areas for further research and suggest caution in interpreting our findings.

Our study identified and addressed potential data leakage issues arising from the random shuffling of data before partitioning. Although we implemented a rigorous train-validation-test splitting strategy to mitigate this risk, it is possible that other forms of data leakage, not considered in this study, could still influence the model's performance. Ensuring data integrity in real-world applications remains a complex challenge, and our approach may not cover all potential scenarios.

Another limitation is that our study primarily used Mean Squared Error and Weighted Accuracy as evaluation metrics. While these metrics provide valuable insights into model performance, they may not capture all aspects of the problem. Future research should explore alternative metrics that better align with the objectives of volatility forecasting, such as economic impact, risk-adjusted returns, or other financial performance measures.

Finally, while our study provides insights into the theoretical and practical challenges of volatility forecasting, the generalizability of our findings to real-world applications remains uncertain. Financial markets are inherently complex and dynamic, and models trained on historical data may not always perform well in live trading environments. Further research is needed to validate our findings across different market conditions and datasets.

10 Future Research

Based on the findings and limitations of our study, we could improve future work as follows:

- Enhancing data collection and preprocessing: To improve the reliability of volatility forecasting models, it is crucial to have access to high-quality, comprehensive datasets. Additionally, implementing rigorous data cleaning and preprocessing procedures will help ensure the integrity and accuracy of the data used for model training and evaluation.
- Optimizing model complexity and regularization: While high-complexity models such as neural networks and gradient boosting models can capture intricate patterns in the data, they are also prone to overfitting. Hyperparameter tuning and model selection should be conducted with care, balancing complexity and performance.
- Exploring alternative feature variables: Future research could incorporate variables such as political uncertainty (example: geopolitical tensions or wars), currency instability (example: inflation fears), market sentiment, and other macroeconomic indicators. Furthermore, adding more detailed information about the options themselves, such as strike price, trading volume or other Greeks could provide additional insights. Our current model, which utilized only five features, may not fully capture the complexity of these dynamics.
- Exploring alternative target variables: Our study highlighted the importance of choosing an appropriate target variable. Future research should explore different target formulations, such as percentage changes in implied volatility, log returns of implied volatility, and classification of volatility movements.
- Diversifying evaluation metrics: Relying solely on traditional metrics like Mean Squared Error may not fully capture the performance of volatility forecasting models. Future studies should consider incorporating alternative metrics that better align with the objectives of financial applications, such as weighted accuracy, economic impact, risk-adjusted returns, and other financial performance measures.
- Validating models in real-world settings: Models trained on historical data may not always perform well in live trading environments. To enhance the generalizability of research findings, authors should validate their models in real-world settings, considering various market conditions and external factors.

11 Conclusion

To conclude, constructing a proper dataset is challenging and requires careful attention, especially when dealing with multiple time dependencies. Ensuring data quality and consistency is crucial for accurate model training and evaluation. In this study, we successfully replicated the results of Cao et al. (2020), demonstrating the feasibility of their approach, provided that data is properly handled. However, we discovered that XGBoost models can often show better results than artificial neural networks

(ANN) while requiring significantly less computational time. For instance, training a neural network might take an hour, whereas an XGBoost model could complete the same task in about a second, making it 3600 times faster.

A critical finding in our study was the significant data leakage problem associated with random shuffling in time series data. This highlights the necessity of being careful in handling time series data to avoid overly optimistic performance estimates. It might be beneficial to use synthetic data for model selection to mitigate data leakage, as synthetic data provides a controlled environment for better validation of model capabilities before deploying them on real-world data.

Our experiments showed that low-complexity models might outperform high-complexity models in real-world applications, as the latter are more prone to exploiting leakage in the training data, leading to under-performance on actual data. Moreover, with the correct data splitting strategy, our models did not achieve satisfactory results on the selected input features, emphasizing the importance of appropriate data handling. These findings add to the observed limitations of complex non-linear methods in forecasting implied volatility reported in previous works (Gunnarsson et al., 2024; Wen et al., 2024).

The choice of the loss function is also critical. The Mean Squared Error (MSE) is a common choice, but it has limitations, particularly when it comes to capturing the true financial impact of predictions. In addition, considering the type of task (whether classification or regression) is essential. In some cases, transforming the problem into a classification task and using weighted accuracy to account for both direction and magnitude of change might provide more meaningful insights and robust trading strategies.

In summary, while neural networks are powerful tools, they require substantial resources and careful handling to avoid pitfalls such as data leakage and inappropriate loss function selection. Further research is needed to explore these aspects in greater detail and validate our findings across a broader range of datasets and model types.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature. This work was financially supported by the Institute of Mathematics of the Universitat Politécnica de Catalunya - Barcelona Tech (IMTech) under their 2024 funding programme.

Data Availability The data and Python codes to reproduce the results in this work are available in GitHub: https://github.com/daou1997/Volatility_Forecasting_DSM

Declarations

Competing interests The authors report there are no competing interests to declare.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Balaneji, F., Maringer, & D. (2022). Quarterly time-series forecasting with neural networks. *IEEE Symposium on Computational Intelligence for Financial Engineering and Economics (CIFEr)*

Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series forecasting. *Journal of the Royal Statistical Society: Series C (Applied Statistics), 61*(4), 541–558.

Cao, J., Chen, J., & Hull, J. (2020). A neural network approach to understanding implied volatility movements. *Quantitative Finance, 20*, 1405–1413.

Gonzalves, S., & Guidolin, M. (2006). Predictable dynamics in the S&P 500 index options implied volatility surface. *The Journal of Business, 79*(3), 1591–1635.

Gunnarsson, E. S., Isern, H. R., Kaloudis, A., Risstad, M., Vigdel, B., & Westgaard, S. (2024). Prediction of realized volatility and implied volatility indices using AI and machine learning: A review. *International Review of Financial Analysis*, 103221

Hull, J., & White, A. (2017). Optimal Delta hedging for options. *Journal of Banking and Finance, 82*, 180–190.

Liu, S., Oosterlee, C. W., & Bohte, S. M. (2019). Pricing options and computing implied volatilities using neural networks. *Risks, 7*(1), 16. https://doi.org/10.3390/risks7010016

Lopez de Prado, M. (2018). *Advances in Financial Machine Learning*, Wiley

Schnaubelt, M. (2019). A comparison of machine learning model validation schemes for non-stationary time series data. *FAU Discussion Papers in Economics*, 11/2019

Skiadopoulos, G., Hodges, S., & Clewlow, L. (2000). The dynamics of the S&P 500 implied volatility surface. *Review of Derivatives Research, 3*, 263–282.

Vrontos, S. D., Galakis, J., & Vrontos, I. D. (2021). Implied volatility directional forecasting: a machine learning approach. *Quantitative Finance, 21*, 1687–1706.

Wen, C., Zhai, J., Wang, Y., & Cao, Y. (2024). Implied volatility is (almost) past-dependent: Linear vs nonlinear models. *International Review of Financial Analysis, 95*, Article 103406.

Zhang, G.P.; Kline, D.M.: Quarterly time-series forecasting with neural networks. *IEEE Transactions on Neural Networks, 18*(6), 1800–1814

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Argimiro Arratia[1] · Mahmoud El Daou[1] · Jan Kagerhuber[2] · Yulia Smolyarova[2]

✉ Argimiro Arratia
argimiro.arratia@upc.edu

Mahmoud El Daou
mahmoud.el.daou@upc.edu

Jan Kagerhuber
jan.kagerhuber@bse.eu

Yulia Smolyarova
yulia.smolyarova@bse.eu

[1] Department of Computer Science, Universitat Politécnica de Catalunya, Barcelona, Spain

[2] Barcelona School of Economics, Universitat Pompeu Fabra, Barcelona, Spain